Unit-2

Exception Handling, Threading, JDBC

UNIT – II

- **Exception Handling, Threading:** Exception Hierarchy, Exception Methods, Catching Exceptions, Multiple catch Clauses, Uncaught Exceptions Java's Built-in Exception. Creating, Implementing and Extending thread, thread priorities, synchronization suspending, resuming and stopping Threads, Multi-threading.

- **JDBC:** Introduction, Drivers and architecture, Connections, statement, result set. Store, retrieve and transaction management.

Exception Handling

- An ***exception*** is an abnormal condition that arises in a code sequence at run time.
- In other words, an exception is a **run-time error**.
- In computer languages that do not support exception handling, errors must be checked and handled manually—typically through the use of error codes, and so on.
- This approach is as cumbersome as it is troublesome.
- Java's exception handling avoids these problems and, in the process, brings run-time error management into the object oriented world.

Exception-Handling Fundamentals

- This is the general form of an exception-handling block:

```
try {
// block of code to monitor for errors
}
catch (ExceptionType1 exOb) {
// exception handler for ExceptionType1
}
catch (ExceptionType2 exOb) {
// exception handler for ExceptionType2
}
// ...
finally {
// block of code to be executed after try block ends
}
```

- Here, *ExceptionType is the type of exception that has occurred.*
- *The remainder of this chapter* describes how to apply this framework.

Exception Types

- All exception types are subclasses of the built-in class **Throwable.**
- Thus, **Throwable** is at the top of the exception class hierarchy.
- Immediately below **Throwable** are two subclasses that partition exceptions into two distinct branches. One branch is headed by **Exception.**
- This class is used for exceptional conditions that user programs should catch.
- This is also the class that you will subclass to create your own custom exception types.
- There is an important subclass of **Exception**, called **Runtime Exception.**
- **Exceptions** of this type are automatically defined for the programs that you write and include things such as **division by zero** and **invalid array indexing.**

- The other branch is topped by **Error,** which defines exceptions that are not expected to be caught under normal circumstances by your program.
- Exceptions of type **Error** are used by the Java run-time system to indicate errors having to do with the run-time environment, itself.
- **Stack overflow** is an example of such an error.

Uncaught Exceptions

- Before you learn how to handle exceptions in your program, it is useful to see what happens when you don't handle them.
- This small program includes an expression that intentionally causes a divide-by-zero error:

```
class Exc0 {
public static void main(String args[]) {
int d = 0;
int a = 42 / d;
}
}
```

- **Example:** Exec1.java
- **Example:** Exec2.java

Multiple catch Clauses

- In some cases, more than one exception could be raised by a single piece of code.
- To handle this type of situation, you can specify two or more **catch clauses,** each catching a different type of exception.
- When an exception is thrown, each **catch** statement is inspected in order, and the first one whose type matches that of the exception is executed.
- After one **catch** statement executes, the others are bypassed, and execution continues after the **try/catch** block.
- The following example traps two different exception types:
- **Example:** MultiCatch.java
- **Example:** SuperSubCatch.java

Nested try Statements

- The **try statement** can be nested. That is, a try statement can be inside the block of another try.
- Each time a **try statement** is entered, the context of that exception is pushed on the stack.
- **If an** inner **try statement** does not have a catch handler for a particular exception, the stack is unwound and the next **try statement's catch handlers** are inspected for a match.
- This continues until one of the **catch statements** succeeds, or until all of the **nested try statements** are exhausted.
- If no **catch statement matches,** then the Java run-time system will handle the exception.
- Here is an example that uses nested **try statements:**
- **Example:** NestTry.java
- **Example:** MetNestTry.java

throw

- It is possible for your program to throw an exception explicitly, using the **throw** statement.
- The general form of **throw is shown here:**

  throw *ThrowableInstance;*
- Here, *ThrowableInstance must be an object of type* ***Throwable or a subclass of Throwable.***
- Primitive types, such as **int or char,** as well as **non-Throwable classes,** such as **String and Object,** cannot be used as **exceptions.**
- There are two ways you can obtain a **Throwable object:**
- using a parameter in a **catch clause,** or creating one with the **new operator.**
- **Example:** DemoThrow.java

throws

- If a **method** is capable of **causing an exception** that it **does not handle**, it **must specify** this behavior so that **callers** of the method can guard themselves against that exception.
- You do this by including a **throws clause** in the method's declaration.
- A **throws clause** lists the types of exceptions that a method might throw.
- This is necessary for all exceptions, except those of type **Error** or **RuntimeException,** or any of their subclasses.
- All other exceptions that a method can throw must be declared in the **throws clause.**
- If they are not, a compile-time error will result.
- This is the general form of a method declaration that includes a **throws clause:**

```
type method-name(parameter-list) throws exception-list
{
// body of method
}
```

Example: ThrowsDemo.java

finally

- When exceptions are thrown, execution in a method takes a rather abrupt, nonlinear path that alters the normal flow through the method.
- Depending upon how the method is coded, it is even possible for an exception to cause the method to return prematurely.
- This could be a problem in some methods.
- For example, if a method **opens a file upon entry** and **closes it upon exit,** then you will not want the **code that closes the file** to be **bypassed** by the **exception-handling mechanism.**
- The **finally** keyword is designed to address this contingency.
- **Example:** FinallyDemo.java

Java's Built-in Exceptions

- Inside the standard package **java.lang,** Java defines several **exception classes.**
- A few have been used by the preceding examples.
- The most general of these exceptions are subclasses of the standard type **RuntimeException.**
- As previously explained, these **exceptions** need not be included in any method's **throws list.**
- In the language of Java, these are called *unchecked exceptions* because the compiler does not check to see if a method handles or throws these exceptions.
- The unchecked exceptions defined in **java.lang** are listed in Table lists.
- Those exceptions defined by **java.lang** that must be included in a method's **throws** list if that method can generate one of these **exceptions** and does not handle it itself.
- These are called ***checked exceptions.***
- *Java defines several other types* of exceptions that relate to its various class libraries.

Exception	Meaning
ArithmeticException	Arithmetic error, such as divide-by-zero.
ArrayIndexOutOfBoundsException	Array index is out-of-bounds.
ArrayStoreException	Assignment to an array element of an incompatible type.
ClassCastException	Invalid cast.
EnumConstantNotPresentException	An attempt is made to use an undefined enumeration value.
IllegalArgumentException	Illegal argument used to invoke a method.
IllegalMonitorStateException	Illegal monitor operation, such as waiting on an unlocked thread.
IllegalStateException	Environment or application is in incorrect state.
IllegalThreadStateException	Requested operation not compatible with current thread state.
IndexOutOfBoundsException	Some type of index is out-of-bounds.
NegativeArraySizeException	Array created with a negative size.
NullPointerException	Invalid use of a null reference.
NumberFormatException	Invalid conversion of a string to a numeric format.
SecurityException	Attempt to violate security.
StringIndexOutOfBounds	Attempt to index outside the bounds of a string.
TypeNotPresentException	Type not found.
UnsupportedOperationException	An unsupported operation was encountered.

TABLE 10-1 Java's Unchecked **RuntimeException** Subclasses Defined in **java.lang**

Exception	Meaning
ClassNotFoundException	Class not found.
CloneNotSupportedException	Attempt to clone an object that does not implement the **Cloneable** interface.
IllegalAccessException	Access to a class is denied.
InstantiationException	Attempt to create an object of an abstract class or interface.
InterruptedException	One thread has been interrupted by another thread.
NoSuchFieldException	A requested field does not exist.
NoSuchMethodException	A requested method does not exist.

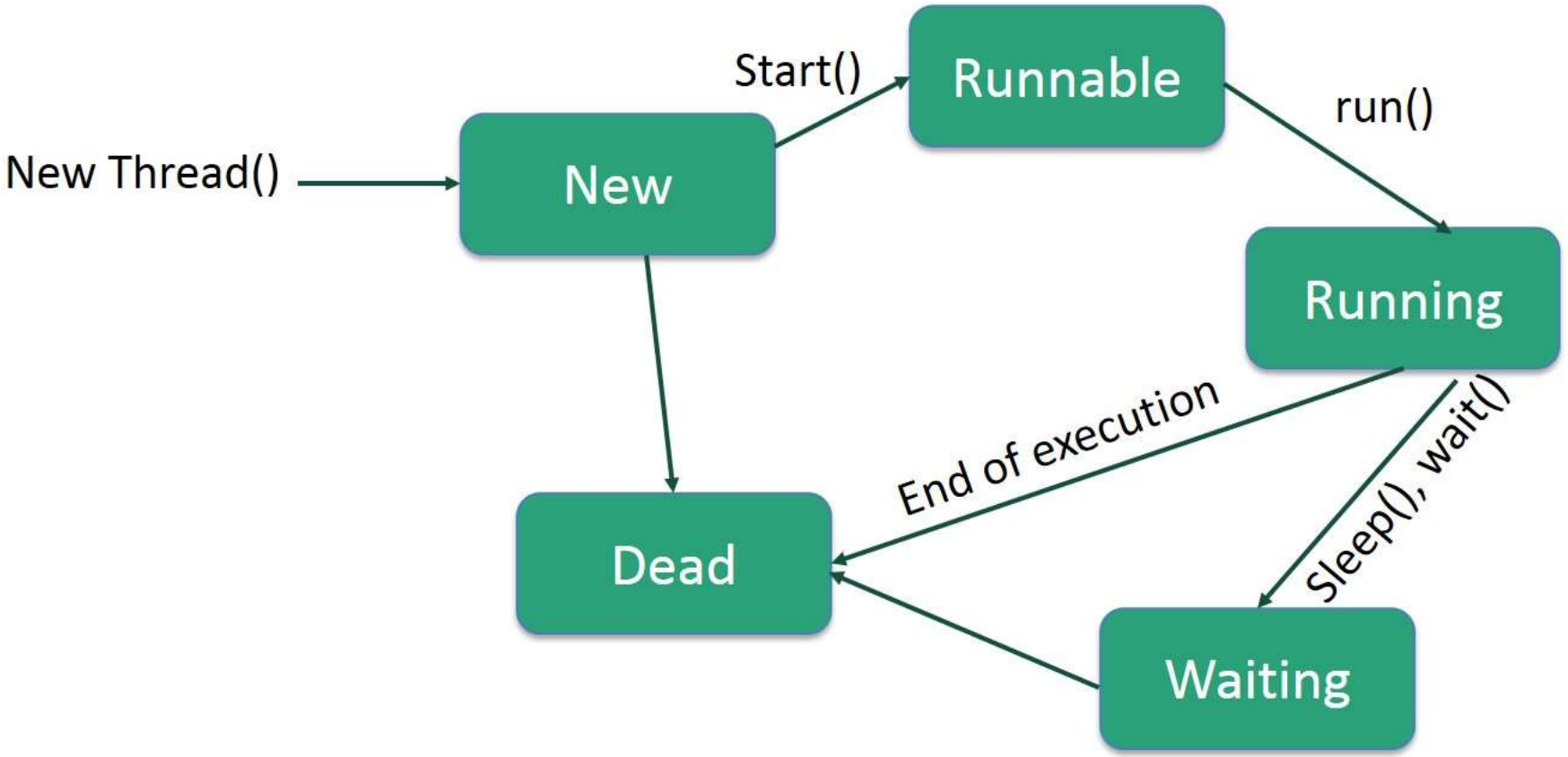
TABLE 10-2 Java's Checked Exceptions Defined in **java.lang**

Java - Multithreading

- **Multithreading** refers to two or more tasks executing concurrently within a single program.
- A thread is an **independent path of execution** within a program.
- Many threads can run concurrently within a program.
- Every thread in Java is **created and controlled** by the **java.lang.Thread class**.
- A Java program can have many threads, and these threads can run concurrently, either asynchronously or synchronously.
- Multithreading has several advantages over Multiprocessing such as;
  - Threads are **lightweight** compared to processes
  - Threads **share the same address space** and therefore can share both data and code
  - **Context switching** between threads is usually **less expensive** than between processes
  - **Cost of thread intercommunication** is relatively low that that of process intercommunication
  - Threads allow different tasks to be performed **concurrently.**

Life Cycle of a Thread

- A thread goes through various stages in its life cycle. For example, a thread is born, started, runs, and then dies.



Following are the stages of the life cycle −

- **New** − A new thread begins its life cycle in the new state. It remains in this state until the program starts the thread. It is also referred to as a **born thread**.
- **Runnable** − After a newly born thread is started, the thread becomes runnable. A thread in this state is considered to be executing its task.
- **Waiting** − Sometimes, a thread transitions to the waiting state while the thread waits for another thread to perform a task. A thread transitions back to the runnable state only when another thread signals the waiting thread to continue executing.
- **Timed Waiting** − A runnable thread can enter the timed waiting state for a specified interval of time. A thread in this state transitions back to the runnable state when that time interval expires or when the event it is waiting for occurs.
- **Terminated (Dead)** − A runnable thread enters the terminated state when it completes its task or otherwise terminates.

Create a Thread by Implementing a Runnable Interface

- If your class is intended to be executed as a thread then you can achieve this by implementing a **Runnable** interface.
- You will need to follow three basic steps –

Step 1

- As a first step, you need to implement a run() method provided by a **Runnable** interface. This method provides an entry point for the thread and you will put your complete business logic inside this method. Following is a simple syntax of the run() method –

```
public void run( )
```

Step 2

- As a second step, you will instantiate a **Thread** object using the following constructor –

```
Thread(Runnable threadObj, String threadName);
```

- Where, *threadObj* is an instance of a class that implements the **Runnable** interface and **threadName** is the name given to the new thread.

Step 3

- Once a Thread object is created, you can start it by calling **start()** method, which executes a call to run( ) method. Following is a simple syntax of start() method −

```
void start();
```

The Main Thread

- When a Java program starts up, one thread begins running immediately.
- This is usually called the ***main thread*** *of your program, because it is the one that is executed when your* program begins.
- The main thread is important for two reasons:
  - It is the thread from which other "child" threads will be spawned.
  - Often, it must be the last thread to finish execution because it performs various shutdown actions.
- Although the main thread is created automatically when your program is started, it can be controlled through a **Thread object.**
- To do so, you must **obtain a reference to it by calling the method currentThread( ),** which is a **public** static member of **Thread.**
- Its general form is shown here:

```
static Thread currentThread( )
```

- **Example:** CurrentThread.java

Creating a Thread

- In the most general sense, you create a thread by instantiating an object of type **Thread.**
- Java defines two ways in which this can be accomplished:
  - You can implement the **Runnable interface.**
  - You can extend the **Thread class, itself.**
- The following example programs look at each method, in turn.
- **Example:** ThreadDemo.java

Extending Thread

- The second way to create a thread is to create a new class that extends **Thread,** and then to create an instance of that class.
- The extending class must override the **run( ) method,** which is the entry point for the new thread. It must also call **start( )** to begin execution of the new thread. Here is the preceding program rewritten to extend **Thread:**
- **Example:** ExtendThread.java

Thread Priorities

- Every Java thread has a priority that helps the operating system determine the order in which threads are scheduled.
- Java thread priorities are in the range between MIN_PRIORITY (a constant of 1) and MAX_PRIORITY (a constant of 10).
- By default, every thread is given priority NORM_PRIORITY (a constant of 5).
- Threads with higher priority are more important to a program and should be allocated processor time before lower-priority threads.
- However, thread **priorities cannot guarantee** the order in which threads execute and are very much **platform dependent**.

Example: HiLoPri.java

Synchronization

- When two or more threads need access to a **shared resource**, they need some way to ensure that the resource will be used by only one thread at a time.
- The process by which this is achieved is called ***synchronization.***
- Key to synchronization is the concept of the monitor (also called a *semaphore).*
- *A monitor* is an object that is used as a mutually exclusive lock, or *mutex. Only one thread can own a* **monitor** at a given time.
- When a thread acquires a lock, it is said to have *entered the monitor.*
- All other threads attempting to enter the locked monitor will be suspended until the first thread *exits the monitor.*
- *These other threads are said to be waiting for the monitor.*
- *A thread* that owns a monitor can reenter the same monitor if it so desires.

Example: Synch.java

Suspending, Resuming, and Stopping Threads

- Sometimes, suspending execution of a thread is useful. For example, a separate thread can be used to display the time of day.
- If the user doesn't want a clock, then its thread can be suspended. Whatever the case, suspending a thread is a simple matter.
- Once suspended, restarting the thread is also a simple matter.
- **Example:** SuspendResume.java

Deadlock in multithreading

A **deadlock** occurs in multithreading when two or more threads are waiting for each other to release **resources**, preventing further execution.

Example of Deadlock

Consider two threads:

1. **Thread-1** locks **Resource-A** and waits for **Resource-B**.
2. **Thread-2** locks **Resource-B** and waits for **Resource-A**.

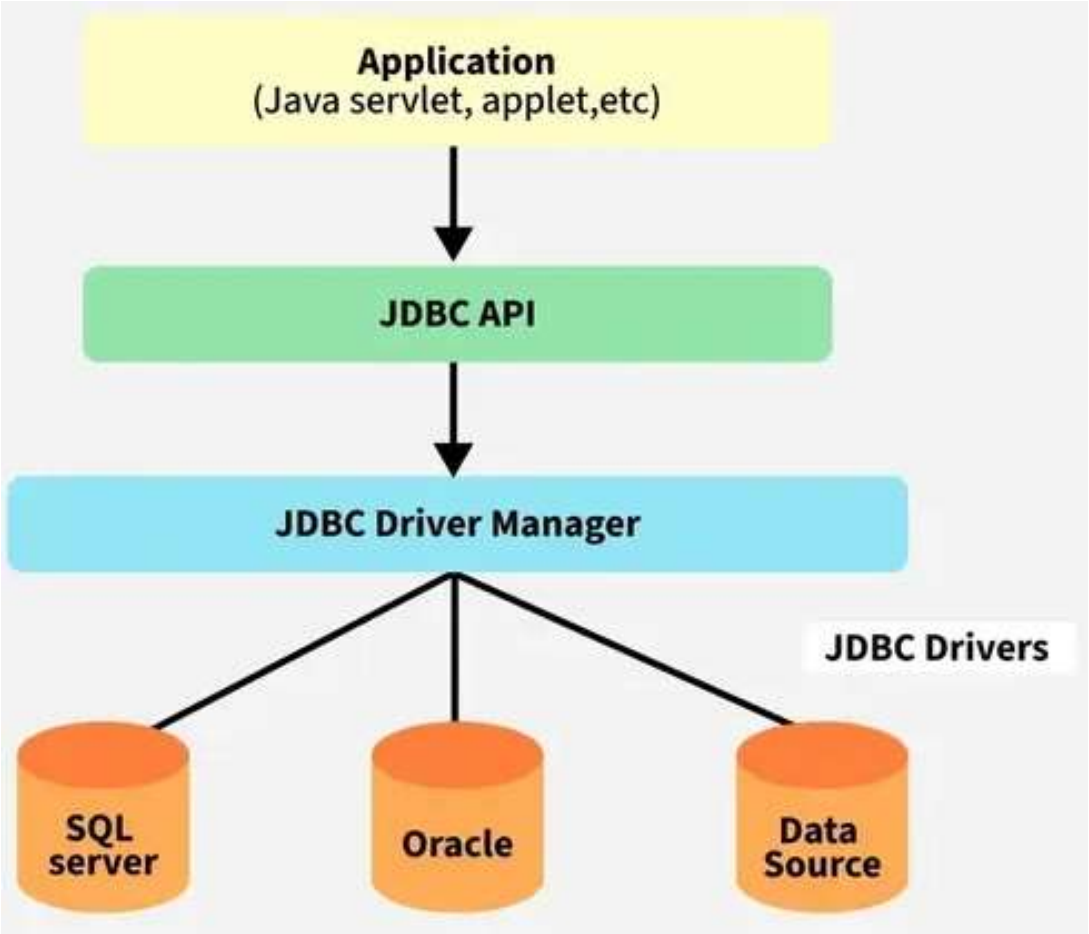
```
class Resource {
    void methodA(Resource res) {
        synchronized (this) {
            System.out.println(Thread.currentThread().getName() + " locked Resource A");
            try { Thread.sleep(100); } catch (InterruptedException e) {}
            synchronized (res) {
                System.out.println(Thread.currentThread().getName() + " locked Resource B");
            }       }   }
```

```
public class DeadlockExample {
    public static void main(String[] args) {
        Resource resA = new Resource();
        Resource resB = new Resource();
        // Thread-1 locks resA first and waits for resB
        Thread t1 = new Thread(() -> resA.methodA(resB), "Thread-1");

        // Thread-2 locks resB first and waits for resA
        Thread t2 = new Thread(() -> resB.methodA(resA), "Thread-2");

        t1.start();
        t2.start();
    }
}
```

Definition of JDBC(Java Database Connectivity)

- JDBC is an API(Application programming interface) used in Java programming to interact with databases. The classes and interfaces of JDBC allow the application to send requests made by users to the specified database.

Architecture of JDBC



Components of JDBC

1. Application: It can be a **Java application or servlet** that communicates with a data source.

2. JDBC API: It allows Java programs to execute SQL(Structured Query Language) queries and get results from the database.

- Some key components of the JDBC API include interfaces such as Driver, ResultSet, RowSet, PreparedStatement, and Connection that help manage various database tasks.
- Classes like DriverManager, Types, Blob, and Clob help manage database connections.

3. DriverManager: It plays an important role in the JDBC architecture. It uses some **database-specific drivers** to effectively connect enterprise applications to databases.

4. JDBC drivers: These drivers handle interactions between the application and the database.

JDBC Architecture

JDBC follows a **two-tier or three-tier architecture**:

1. **Two-Tier Architecture** (Direct Communication)
   - The Java application communicates directly with the database using JDBC.
   - The **JDBC Driver** sends queries to the database and retrieves results.
   - Used in standalone applications.
2. **Three-Tier Architecture** (Via Middleware)
   - The Java application communicates with a **middleware** (e.g., application server).
   - The middleware interacts with the database using JDBC.
   - Used in enterprise applications (e.g., web applications).

How JDBC Facilitates Database Connectivity in Java Applications

1. **Database Independence:**
   - JDBC provides a common API to connect with different databases (MySQL, Oracle, PostgreSQL, etc.).
2. **Easy Query Execution:**
   - Developers can execute **SQL(Structured Query Language)** queries using simple Java code.
3. **Supports Transactions:**
   - JDBC allows transaction management (commit, rollback) for reliable database operations.
   - A transaction is a group of operations treated as a single unit (Either all **succeed (commit) or all fail (rollback))**
4. **Secure & Efficient:**
   - PreparedStatement prevents **SQL injection attacks**.
   - CallableStatement improves **stored procedure execution**.

JDBC Drivers

- **JDBC drivers** are client-side adapters (installed on the client machine, not on the server) that convert requests from Java programs to a protocol that the DBMS can understand. **JDBC drivers** are the software components that implement interfaces in JDBC APIs to enable Java applications to interact with the database.

There are four types of JDBC drivers defined by Sun Microsystems that are mentioned below:

- Type-1 driver or JDBC-ODBC bridge driver
- Type-2 driver or Native-API driver
- Type-3 driver or Network Protocol driver
- Type-4 driver or Thin driver

1. JDBC-ODBC Bridge Driver (Type 1)

- This is the **oldest and least efficient** driver. It uses the ODBC (Open Database Connectivity) driver to connect to the database, which acts as an intermediary between the Java application and the database. The JDBC-ODBC bridge converts JDBC calls into ODBC calls, and then the ODBC driver translates them into database-specific calls.
- **Working:**
  - The Java application calls JDBC methods.
  - The JDBC-ODBC bridge translates them into ODBC calls.
  - The ODBC driver communicates with the database.
- **Advantages:**
  - Easy to use with existing ODBC data sources.
  - Supports many databases because ODBC drivers are available for various databases.
- **Disadvantages:**
  - **Slower performance** because of the extra layer (ODBC bridge).
  - It is **platform-dependent** as it relies on ODBC, which is not available on all systems.
  - **Deprecated** in newer versions of Java (Java 8 and beyond).

2. Native-API Driver (Type 2)

- This driver uses the database's native API (such as client libraries or other specific database protocols) to interact with the database. It directly calls the database's client libraries to execute SQL queries.
- **Working:**
  - The Java application calls JDBC methods.
  - The Type 2 driver converts the JDBC calls into native database calls.
  - The database's native **client software or library** processes these calls.
- **Advantages:**
  - Better performance than the Type 1 driver because it avoids the ODBC layer.
  - Provides more direct access to the database.
- **Disadvantages:**
  - **Platform-dependent** because each database has its own native client library.
  - Requires a **specific client library** for each database, which increases complexity.
  - **Limited portability** as the application will only work with specific databases that have compatible client libraries.

3. Network Protocol Driver (Type 3)

- This driver communicates with the database through a **middleware server** that translates the JDBC calls into database-specific calls. The middleware server uses a common network protocol to communicate with the Java application and the database.
- **Working:**
  - The Java application sends JDBC calls to the Type 3 driver.
  - The Type 3 driver forwards the requests to the **middleware server** using a common protocol.
  - The **middleware server processes** the requests and sends them to the database.
  - The server then returns the results to the driver, which sends them to the Java application.
- **Advantages:**
  - **Platform-independent** because the driver uses a common protocol, and there is no need for native database client libraries.
  - Can **connect to any database** that supports the protocol, making it highly flexible.
  - **Good performance** since the middleware can manage **multiple databases** simultaneously.
- **Disadvantages:**
  - Requires an **additional server**, which adds complexity and potential bottlenecks.
  - **Slightly slower** than direct connections due to the middleware layer.

4. Thin Driver (Type 4)

- This is a **pure Java driver** that directly converts JDBC calls into database-specific network protocol calls. The Type 4 driver doesn't require any native libraries or additional server layers—it communicates directly with the database using its proprietary protocol.
- **Working:**
  - The Java application calls JDBC methods.
  - The Type 4 driver translates the **JDBC calls** into the **database's native protocol**.
  - The database directly receives and processes the requests.
  - Results are returned back through the same path.
- **Advantages:**
  - **Platform-independent** because it's implemented entirely in Java and doesn't require any native libraries.
  - **High performance** because it communicates directly with the database and has minimal overhead.
  - **Easy to deploy**—the driver is lightweight, and there's no need for additional configuration or external libraries.
- **Disadvantages:**
  - **Tightly coupled** to the database's protocol, meaning the driver needs to be updated when the database's protocol changes.
  - A **separate driver** is needed for each type of database, as each database has its own proprietary protocol.

Accessing a database in Java using JDBC

To access a database in Java using JDBC, the following steps are followed:

- **Loading the JDBC Driver**: First, load the database-specific driver using:

```
Class.forName("com.mysql.cj.jdbc.Driver");  // For MySQL
```

- **Establishing a Connection**: Create a connection to the database using:

```
Connection con = DriverManager.getConnection("jdbc:mysql://localhost:3306/mydatabase", "username", "password");
```

- **Creating a Statement**: To execute SQL queries, create a Statement object:

```
Statement stmt = con.createStatement();
```

- **Executing Queries**: Use executeQuery() for SELECT statements or executeUpdate() for INSERT, UPDATE, or DELETE:

```
ResultSet rs = stmt.executeQuery("SELECT * FROM employees");
```

- **Processing the Results**: Use a ResultSet to access data:

```
while (rs.next()) {
    System.out.println("ID: " + rs.getInt("id") + ", Name: " +
rs.getString("name"));
}
```

- **Closing the Connection**: Finally, close the resources:

```
rs.close();
stmt.close();
con.close();
```

- This allows you to connect to the database, run SQL queries, and handle the results.

In JDBC (Java Database Connectivity), there are three types of statements used to interact with a database:

1. **Statement**
   - Used to execute simple SQL queries (static SQL).
   - Not optimized for repeated execution.
   - Example: stmt.executeQuery("SELECT * FROM employees");
2. **PreparedStatement**
   - Used for executing parameterized queries (dynamic SQL).
   - More efficient than Statement because it precompiles queries.
   - Prevents SQL injection attacks.
   - Example: psmt = con.prepareStatement("SELECT * FROM employees WHERE id = ?");
3. **CallableStatement**
   - Used to call stored procedures in the database.
   - Optimized for executing complex business logic on the database server.
   - Example: CallableStatement cs = con.prepareCall("{call getEmployeeDetails(?)}");